

■ RAG INTERVIEW PREPARATION

100 RAG Interview Questions

The Complete Study Guide for AI Engineers · 2026 Edition

For software engineers and students prepping for AI Engineer interviews where RAG comes up — roughly 36% of all AI Engineer roles. RAG is the #1 most-asked AI engineering topic in 2026. Don't memorize: try to answer each question out loud in under 60 seconds, then read the answer to see what you missed.

100

QUESTIONS

05

CATEGORIES

70+

TO BE INTERVIEW-READY

CURATED & PRESENTED BY

DIPANSHU.DECODED

Backend Engineering · AI/ML Engineering · Career Guidance

DIPANSHU.DECODED · 100 QUESTIONS · FULL ANSWERS

A structured question bank covering everything RAG interviews probe for — fundamentals, chunking & embeddings, vector databases & retrieval, evaluation, and production-grade advanced patterns. If you can confidently answer 70+ out of 100, you're ready for senior RAG interviews. Hit 90+, and you're in the top 5%.

01 / RAG Fundamentals

Questions 1–20

Q1 What is RAG and what problems does it actually solve?

RAG (Retrieval-Augmented Generation) is a pattern where an LLM is given relevant context retrieved from an external knowledge base at inference time, instead of relying only on what it learned during training. It solves three core problems: knowledge cutoffs, hallucinations, and verifiability. RAG turns a generic LLM into a domain-aware system without retraining.

Q2 Walk me through the components of a RAG pipeline end-to-end.

Ingestion (offline): documents are loaded, chunked, embedded, and stored in a vector database with metadata. Inference (online): the user query is embedded, top-k chunks are retrieved (often reranked), injected into the prompt, and the LLM generates a grounded answer. Production systems add query rewriting, hybrid search, reranking, citation enforcement, and eval hooks on top.

Q3 When would you choose RAG over fine-tuning?

Choose RAG when data changes frequently, you need verifiable sources, or the corpus is too large to fit a fine-tuning dataset. Choose fine-tuning to teach style or behavior, or when latency/cost demands a smaller specialized model. Most production systems use RAG for facts and fine-tuning for style — they're complements.

Q4 RAG vs longer context windows — when does which win?

Long context wins for small, fixed corpora that fit entirely in the window. RAG wins for large, frequently changing corpora and cost-sensitive use cases, since dumping irrelevant context wastes tokens and degrades quality (lost-in-the-middle). The sweet spot is hybrid: RAG narrows to the top chunks, then a long-context model reasons over them.

Q5 What are the failure modes of a basic RAG system?

(1) retrieval misses; (2) chunking too small or large; (3) the LLM ignores retrieved context; (4) the LLM hallucinates grounded answers; (5) query-document vocabulary mismatch; (6) outdated index. Every production RAG system needs telemetry on each of these.

Q6 Explain the difference between extractive and generative QA.

Extractive QA pulls an exact answer span verbatim — high precision, no hallucination risk, but rigid. Generative QA uses an LLM to synthesize an answer from retrieved context — flexible, handles multi-document reasoning, but can hallucinate. Modern RAG is generative QA with constraints like citation and refusal.

Q7 When is RAG the wrong tool for the job?

When the question needs no external knowledge, the data is highly structured (SQL fits better), you need precise computation, the corpus fits in context anyway, latency must be sub-100ms, or the task needs reasoning across the whole corpus rather than a few documents.

Q8 What is the role of embeddings in RAG?

Embeddings are dense vectors capturing semantic meaning — similar meanings map to nearby vectors. Retrieval is finding document vectors closest to the query vector, usually by cosine similarity. Embedding model quality is one of the biggest determinants of retrieval quality.

Q9 What is the difference between RAG and search?

Traditional search returns documents for a human to read. RAG retrieves chunks and uses an LLM to synthesize a grounded natural-language answer. Search optimizes relevance ranking; RAG additionally needs faithfulness, citation, and synthesis.

Q10 What's the difference between a retriever and a reranker?

A retriever does fast first-pass filtering (vector similarity) returning top-k candidates. A reranker scores those candidates more precisely with a heavier model (often cross-encoder) and reorders to top-N. Retrievers optimize recall; rerankers optimize precision.

Q11 Can RAG work without an LLM?

Sort of — extractive QA achieves the retrieval half without generation. But “RAG” specifically implies the generation/synthesis step. Without an LLM combining retrieved context into an answer, you have search or extractive QA, not RAG.

Q12 What does “grounding” mean in RAG?

Grounding ties the LLM's response to specific, verifiable source material. A grounded answer can be traced back to a source. Strong grounding requires both providing relevant context and enforcing that the model actually uses it.

Q13 What is “context stuffing” and why is it a problem?

Cramming as much retrieved content into the prompt as possible. Problems: cost and latency scale with tokens, lost-in-the-middle effects, and irrelevant context can degrade answer quality. Better: retrieve more, rerank tightly, pass only the top 3–5 chunks.

Q14 What's the “lost in the middle” problem?

LLMs pay disproportionate attention to the start and end of context, with accuracy dropping for information in the middle. Practical implication: order matters — put the most relevant chunks first or last, not buried in the middle.

Q15 What is a “chunk” in RAG?

A unit of text, typically 200–1000 tokens, that gets independently embedded and indexed. Documents are split into chunks because embedding a whole document is too coarse. The chunk is the atomic unit of retrieval.

Q16 What metadata should you store with each chunk?

At minimum: document ID, source path, chunk index, offsets, ingestion timestamp, access-control tags. Beyond that: title, section/heading, author, date, language, document type, and domain-specific tags for filtering.

Q17 What's the difference between sync and async RAG?

Sync: query → retrieve → generate → return, blocking the user throughout. Async/streaming: retrieval still blocks, but generation streams tokens as produced. In production you almost always want streaming generation to keep perceived latency low.

Q18 What's a “context window budget” and how do you manage it?

The total tokens the LLM can process per request, split across system prompt, history, retrieved context, and output. When it tightens, you drop shorter history, fewer/smaller chunks, or truncate output.

Q19 What is the difference between recall and precision in RAG retrieval?

Recall: of all truly relevant chunks, how many did we return? Precision: of the chunks we returned, how many were relevant? They trade off — wider retrieval improves recall but hurts precision; reranking casts a wide net then filters tight.

Q20 What's a “naive RAG” and how does it differ from production RAG?

Naive RAG: chunk → embed → store → embed query → top-k → stuff in prompt → generate. Production RAG adds query understanding, hybrid retrieval, reranking, metadata filtering, multi-step retrieval, citation enforcement, evals, caching, and observability.

02 / Chunking & Embeddings

Questions 21–40

Q21 How do you pick the right chunk size for your use case?

Start with 500 tokens, 50-token overlap. Adjust for document structure, embedding model context length, query specificity, and downstream context budget. The honest interview answer: start at 500/50, evaluate on a golden set, and tune empirically.

Q22 What chunking strategies do you know? When to use which?

Fixed-size (simple baseline), recursive (splits on paragraph/sentence/word, good default), semantic (splits on similarity drops, best quality), document-aware (splits on headers/tags, best for structured docs), parent-child (small chunks for retrieval, large for generation context).

Q23 What is semantic chunking and when is it worth the cost?

Computes sentence embeddings and creates a boundary where adjacent-sentence similarity drops below a threshold. Worth it for loosely structured documents, very specific queries, or precision-critical domains like medical/legal. Not worth it for well-structured docs.

Q24 What is chunk overlap and why do you need it?

The last N tokens of chunk i are repeated as the first N tokens of chunk i+1, so relevant info on a boundary isn't missed by either chunk. Typical overlap is 10–20% of chunk size, e.g. 50–100 tokens for 500-token chunks.

Q25 How do you handle tables in chunking?

Don't split tables mid-row. Options: keep the whole table as one chunk, convert to markdown/CSV-like text, treat each row as its own chunk with the header repeated, or store the table as metadata alongside surrounding narrative text.

Q26 How do you handle code in chunking?

Don't split mid-function — use language-aware or AST-based splitters (e.g. tree-sitter) that preserve function/class boundaries and keep imports with the code they apply to. Also consider embedding signatures and docstrings separately.

Q27 How do you handle images and figures in chunking?

Use a multimodal embedding model to embed images directly, run image captioning and treat the caption as chunk content, or keep captions/surrounding text and ignore the image itself. Production systems often caption during ingestion and index that.

Q28 How do you pick an embedding model for production?

Evaluate MTEB performance on your task type, domain match, dimensions, cost/latency (hosted vs. self-hosted), and multilingual needs. Always benchmark 2–3 candidates on your own retrieval eval set before committing.

Q29 When would you train or fine-tune your own embeddings?

When off-the-shelf embeddings systematically miss your domain's semantics, you have at least a few thousand labeled query-document pairs, and general-purpose embeddings underperform despite tuning everything else. Hybrid search + reranking often gets 80% of the gain without training.

Q30 What's the difference between bi-encoders and cross-encoders?

Bi-encoders embed query and document independently and compare via similarity — fast, used for retrieval. Cross-encoders take query and document together and output a relevance score directly — more accurate but slow, used for reranking.

Q31 What is the “parent-document retrieval” pattern?

Index small chunks for precise retrieval, but pass the larger parent section to the LLM when a chunk is retrieved. Gives retrieval precision of small chunks with contextual completeness of large ones.

Q32 What are sparse embeddings (BM25, SPLADE)?

High-dimensional vectors where most values are zero, with non-zero values for terms present. BM25 is classical keyword matching with TF-IDF weighting; SPLADE is a neural sparse model. Strong on exact matches and rare terms/named entities.

Q33 What does dimensionality mean for embeddings?

The length of the vector representing each text. Higher dimensions capture more nuance but cost more storage/compute. 384–768 is enough for most apps; 3072 is for cases where every percentage point of recall matters.

Q34 What is Matryoshka Representation Learning (MRL)?

A training technique where prefixes of the full embedding vector are themselves valid lower-dimensional embeddings. You can truncate a 3072-dim MRL embedding to 768 dims and still get a usable, slightly less accurate embedding. Used in OpenAI's text-embedding-3.

Q35 What's the difference between sentence embeddings and word embeddings?

Word embeddings give one vector per word regardless of context. Sentence embeddings give one context-aware vector per sentence/chunk using transformer models. RAG uses sentence-level embeddings exclusively.

Q36 How do you handle multilingual RAG?

Use a multilingual embedding model that embeds all languages into a shared space (the modern default), translate to a pivot language, or maintain per-language indexes with language detection at query time.

Q37 What is “instruction-tuned” embedding?

Some embedding models accept an instruction prefix before embedding, so the same text gets different embeddings depending on the task. In RAG, queries and documents typically get different instructions.

Q38 How do you embed long documents?

You don't embed them whole — embedding models have token limits and long inputs become semantically meaningless averages. Chunk first, then embed each chunk; for document-level representations, embed the title plus first N tokens or average chunk embeddings.

Q39 What's a typical embedding latency budget?

Hosted APIs: 50–200ms per query embedding, with much higher batch throughput. Self-hosted on GPU: under 50ms per query. Query embedding shouldn't be your bottleneck — if it is, you're probably not batching.

Q40 What is embedding drift and how do you detect it?

The distribution of produced embeddings shifts over time due to model updates or new document types. Detect via similarity score distributions, retrieval quality on a fixed eval set, and score distribution shifts. Fix by re-embedding or pinning a model version.

03 / Vector DBs & Retrieval

Questions 41–60

Q41 Compare pgvector, Pinecone, Weaviate, Qdrant. When does which win?

pgvector: free, transactional, great under ~10M vectors if you already use Postgres. Pinecone: hosted, hands-off, expensive at scale. Weaviate: open-source, strong hybrid search. Qdrant: open-source, fastest self-hosted, best performance per dollar.

Q42 What is HNSW and why is it the default index for vector DBs?

A graph-based approximate nearest neighbor index with multiple layers for fast navigation. Default because of excellent recall at very low latency, tunable parameters, and no retraining needed on inserts.

Q43 What's the difference between exact and approximate nearest neighbor search?

Exact (brute force) computes similarity against every vector — fine to ~100K vectors. Approximate (HNSW, IVF, ScaNN) trades a little accuracy for huge speed at scale. Production RAG always uses ANN; exact is for evaluation only.

Q44 Dense vs sparse retrieval — when do you need both?

Dense is strong on semantic similarity and paraphrases; sparse is strong on exact matches and rare terms/identifiers. You need both when queries mix conceptual questions with specific identifiers — most production systems use hybrid search.

Q45 What is hybrid search and how would you implement it?

Combines dense and sparse retrieval, merged via Reciprocal Rank Fusion (most common, parameter-free), weighted score combination (needs normalization), or a vector DB with native hybrid support.

Q46 What is Reciprocal Rank Fusion (RRF)?

Combines multiple ranked lists by summing $1/(k + \text{rank})$ for each result across lists ($k \approx 60$). Robust to score-scale differences since only ranks matter, simple, and consistently strong across domains.

Q47 How does reranking work and when is it worth the latency?

Cross-encoders score top-k retrieved candidates more precisely than embedding similarity and reorder to top-N. Worth the 50–200ms latency almost always in production, since precision typically matters more than the small delay.

Q48 How do you implement metadata filtering at scale?

Pre-filter (filter then search, expensive if unselective), post-filter (search then filter, may under-return), or filter-aware ANN indexes that integrate filtering into vector search — best of both, offered natively by most modern vector DBs.

Q49 How do you handle multi-tenant data isolation in a vector DB?

One namespace per tenant (strong isolation, costly at scale), metadata-based isolation in a shared index (cheaper, riskier), or tiered isolation for top customers. Shared-with-filter is most common, with rigorous filter testing.

Q50 What's the difference between cosine similarity, dot product, and Euclidean distance?

Cosine measures angle, ignoring magnitude — standard for embeddings. Dot product is cosine times magnitude, faster if vectors are pre-normalized. Euclidean is straight-line distance. For L2-normalized embeddings, cosine and dot product rank identically.

Q51 How do you handle large-scale ingestion (millions of documents)?

Batch embedding calls, parallelize across workers, use bulk-insert APIs (10–100x faster), stage embeddings to S3/Parquet for retry-ability, and pre-compute offline rather than during a deploy.

Q52 What is “vector quantization” and when do you use it?

Compresses embeddings from float32 to float16, int8, or binary for massive storage/speed gains at minor recall cost. Use at 10M+ vectors or when storage/latency is the bottleneck.

Q53 What's the difference between IVF, HNSW, and ScaNN indexes?

IVF clusters vectors and searches nearby partitions — fast but recall depends on cluster quality. HNSW is graph-based with high recall — the default choice. ScaNN uses anisotropic quantization, used at Google scale.

Q54 How do you handle real-time updates to a vector index?

Most vector DBs support live upsert/delete with eventual consistency. A common pattern separates a small “hot” index for recent writes from a bulk “cold” index, querying and merging both.

Q55 What's a “namespace” or “collection” in a vector DB?

A logical container separating one set of vectors from another within the same DB instance, used for multi-tenancy, versioning, or topic separation — like a database schema.

Q56 How do you do “search-as-you-type” with vector search?

Vector search isn't great for incremental keystrokes. Solutions: debounce and only embed periodically, use traditional prefix search for autocomplete and reserve dense search for finalized queries, or cache common partial queries.

Q57 What is “cold-start” in vector search and how do you handle it?

Serving queries before the index is warmed in memory, causing slow first queries. Mitigated by preloading the index at deploy, running synthetic warm-up queries, and using provisioned concurrency for serverless.

Q58 How does GPU help with vector search?

GPUs dramatically speed up brute-force similarity and ANN index construction via batched matrix multiplies. Worth it above 100M vectors or high QPS; most smaller systems run fine on CPU with HNSW.

Q59 What is “query expansion” in retrieval?

Enriching the query with related terms before retrieval, via synonyms, LLM-based rephrasing (most common today), or pseudo-relevance feedback. Improves recall on short or vague queries, at some latency cost.

Q60 What is “query decomposition” in retrieval?

Splitting a complex query into sub-questions, retrieving for each, and combining results — typically done by an LLM. Used for multi-hop questions where single-step retrieval misses relevant context.

04 / Evaluation

Questions 61–80

Q61 How do you evaluate a RAG system end-to-end?

Retrieval evaluation (precision@k, recall@k, MRR, NDCG against a labeled set) plus generation evaluation (faithfulness, answer relevance, answer correctness). Production adds user satisfaction, citation click-through, and escalation rate.

Q62 What metrics measure retrieval quality?

Precision@k, recall@k, MRR (penalizes burying the right answer), and NDCG (the gold standard with graded relevance). Recall@10 and MRR are the most commonly tracked in production.

Q63 What metrics measure generation quality?

Faithfulness (are claims supported by context?), answer relevance, answer correctness, context precision, and context recall. Faithfulness is the most important — an unfaithful answer is a hallucination.

Q64 How do you build a golden dataset for RAG?

Collect real queries, have annotators identify relevant chunks, write ideal answers, and tag failure modes. Aim for 100–300 high-quality examples; synthetic generation is fine for bootstrapping but should be human-validated.

Q65 What is LLM-as-judge for RAG and what are its limits?

Using a powerful LLM to score outputs against criteria. Limits: bias toward its own model family, run-to-run inconsistency, cost at scale, reasoning errors, and position bias. Mitigate with rotation, chain-of-thought, and human calibration.

Q66 How do you detect retrieval drift in production?

Track similarity score distributions, query-document overlap shifts, no-result rate, citation click-through, and periodic eval on a fixed golden set. Alert on moves beyond 2 std dev from baseline.

Q67 How do you regression-test RAG in CI/CD?

Maintain a golden set in your repo, run it on every pipeline-affecting PR, compare metrics to main and fail on regression, cache golden-set embeddings for speed, and log per-question diffs for reviewers.

Q68 What is “context precision” and how does it differ from precision@k?

Precision@k is pure retrieval relevance. Context precision is generation-aware: of retrieved chunks, how many were actually used in the answer. Low context precision with high precision@k signals hallucination despite good retrieval.

Q69 What is “context recall” and why does it matter?

Whether all information needed to answer the question was present in retrieved chunks. Low context recall means even a perfect LLM can't fully answer — it's a retrieval problem, not a generation one.

Q70 How do you measure faithfulness?

An LLM judge checks whether each claim in the answer is supported by retrieved context; the score is the fraction of supported claims. A faithfulness score below 0.85 is usually a red flag.

Q71 What's the difference between online and offline evaluation?

Offline: a fixed golden dataset in CI, fast and deterministic but limited. Online: real production traffic with user feedback and A/B tests, catching long-tail issues offline misses but noisier. Production needs both.

Q72 How do you A/B test RAG changes safely?

Shadow traffic (zero risk, slow signal), canary rollout (route a small percent, ramp if healthy), or interleaving (side-by-side comparison). Always log metadata for attribution and have an automated rollback trigger.

Q73 How do you evaluate “I don't know” responses?

Build an explicit abstain set of questions the system should refuse. Score refusal accuracy and answered-question quality separately, penalizing both overcautious and overconfident behavior.

Q74 What is “groundedness” and how do you measure it?

How anchored the answer is in retrieved context, essentially equivalent to faithfulness. Measured by labeling each statement as supported, contradicted, or not in context.

Q75 How do you handle subjective or open-ended questions in eval?

Use rubric-based LLM-as-judge scoring on criteria like completeness and clarity, and prefer pairwise preference over absolute scoring. Human evaluation remains the gold standard for calibration.

Q76 What tools do you know for RAG evaluation?

RAGAS, TruLens, DeepEval, Arize Phoenix, LangSmith, and Langfuse. Most production systems combine one observability tool with custom eval scripts.

Q77 How do you evaluate citations?

Citation precision (do citations support their claims), citation recall (are citable claims actually cited), and citation correctness (do citations point to the right source). Critical for legal, medical, and enterprise RAG.

Q78 How do you set up human-in-the-loop evaluation?

Sample interesting cases (low confidence, refusals, thumbs-down), use a side-by-side review UI, calibrate inter-rater agreement, and feed failures back into the golden set on a regular review cadence.

Q79 What's a "smoke test" for RAG and what should be in it?

A fast 10–20 query set run on every deploy: one per major topic, an out-of-scope query, an adversarial/injection query, a citation-required query, and a multi-hop query — under 60 seconds total.

Q80 How do you measure "user satisfaction" in RAG without explicit ratings?

Implicit signals: engagement (citation clicks, scrolling), follow-up/rephrase rate, abandonment, copy events, and escalation to a human — combined into a proxy score validated against explicit ratings.

05 / Production & Advanced RAG

Questions 81–100

Q81 How do you reduce hallucinations in a RAG system?

Strong system prompt constraints, forced citations, refusal few-shot examples, output validation against cited chunks, faithfulness eval in CI, lower temperature, and better retrieval — most hallucinations are retrieval failures.

Q82 How do you force the LLM to cite sources reliably?

Assign stable chunk IDs in the prompt, require inline citations via system prompt and few-shot examples, use structured JSON outputs, and post-process to verify cited chunks exist — combined for >95% reliability.

Q83 What is HyDE (Hypothetical Document Embeddings)?

Instead of embedding the query, ask an LLM to write a hypothetical answer and embed that for search — since answers match document vocabulary/style better than questions do. Useful for short or vague queries, at the cost of an extra LLM call.

Q84 What is query rewriting and when do you need it?

Transforming the raw query into a better retrieval query via an LLM: expanding acronyms, adding conversation context, clarifying pronouns, or decomposing multi-part questions. Needed for conversational or vague queries.

Q85 What is multi-query retrieval?

Generating multiple query variations, retrieving for each, and merging via RRF — catching documents that a single phrasing would miss, at the cost of extra LLM and retrieval calls.

Q86 How do you handle multi-hop / multi-document questions?

Query decomposition into sub-questions, iterative retrieval that uses intermediate answers to formulate new queries, self-RAG, or graph-based RAG. Decomposition and iterative retrieval are most common in production.

Q87 What is “self-RAG”?

A pattern where the LLM decides at each step whether to retrieve again, whether context is relevant, and whether its answer is well-supported, using control tokens during generation — more autonomous but costlier and less predictable.

Q88 What is “agentic RAG”?

Treats retrieval as a tool the agent can call repeatedly with different strategies, useful for complex or multi-source queries spanning structured and unstructured data, at higher cost and harder evaluation.

Q89 What is “RAG fusion”?

Generate multiple query variations, retrieve for each, fuse with RRF, and pass top results to the LLM — combining multi-query retrieval with rank fusion for consistent recall gains on ambiguous queries.

Q90 How do you handle conversational RAG (follow-up questions)?

Bounded conversation memory plus query rewriting that turns the latest turn into a standalone query using history before retrieval, since standalone queries embed far better than fragments.

Q91 How do you manage cost and latency in production RAG?

Exact-match and semantic caching, model routing by query difficulty, tighter context via aggressive reranking, streaming, prompt compression, embedding caching, batching, and tracking cost per query.

Q92 What is semantic caching?

Returning a cached response when a new query is semantically similar (above a similarity threshold) to a previously cached one, not just an exact match — saves cost but risks staleness and false hits if the threshold isn't tuned.

Q93 Design a production RAG system for a 10M-document corpus.

Parallel ingestion with language-aware chunking and batch embedding into an HNSW index (Qdrant/Weaviate); hybrid retrieval with RRF and cross-encoder reranking; tiered generation models with forced citations and output validation; full observability and a large golden-set eval running in CI.

Q94 How do you handle PII in a RAG pipeline?

Detect and redact/pseudonymize PII at ingestion, avoid logging PII at query time, encrypt at rest with restricted access, redact outputs post-generation, and support deletion-on-request for compliance.

Q95 How do you implement prompt injection defenses in RAG?

Treat retrieved context as untrusted, restate system instructions after context (sandwich pattern), validate outputs against the user's actual question, use structured outputs, flag suspicious chunks, and eval against injection attempts in CI.

Q96 How do you build RAG over structured data (databases)?

Use Text-to-SQL rather than pure vector RAG — the LLM generates SQL, optionally informed by vector-retrieved schema documentation. For mixed needs, build a tool-using agent that can call both vector search and SQL.

Q97 What is “long-context RAG” and how does it differ from regular RAG?

Retrieves many more chunks (50–100) and relies on large-context models to find what's relevant themselves — simpler and more robust to retrieval errors, but costlier and still subject to lost-in-the-middle effects.

Q98 What is a “fact-checker” pattern in RAG?

A second LLM call verifies each claim in the generated answer against retrieved context, triggering regeneration, human review, or abstention on unsupported claims — used in high-stakes domains despite the extra cost.

Q99 How do you version a RAG system?

Version prompts (git, semver), pin model versions, treat chunking parameters as part of the version, version vector DB collections for safe rollouts, and track which version each eval metric was measured on.

Q100 What's the future of RAG (as of 2026)?

Long-context models reducing retrieval aggressiveness for short corpora, multimodal RAG, agentic RAG treating retrieval as one tool among many, personalized retrieval, better synthetic eval, and end-to-end trained retrieval. RAG isn't going away — it's getting more sophisticated.

How to Use This Guide

Week 1: Read all 100 questions, mark which you can answer cold.

Week 2: Deep-study your weak categories. For every gap, build a small project.

Week 3: Mock interviews with a friend using this guide.

Week 4: Re-read just the answers — by now they should feel obvious.